

DataFrame 中处理缺失值的常见方法及其适用场景

处理缺失值的完整指南

首先：检测缺失值

代码块

```
1  import pandas as pd
2  import numpy as np
3
4  # 创建包含缺失值的示例数据
5  df = pd.DataFrame({
6      '姓名': ['张三', '李四', '王五', '赵六', '钱七'],
7      '年龄': [25, np.nan, 30, np.nan, 28],
8      '收入': [5000, 6000, np.nan, 8000, np.nan],
9      '城市': ['北京', '上海', np.nan, '广州', '深圳'],
10     '评分': [np.nan, 4.5, 3.8, np.nan, 4.2]
11 })
12
13 print("原始数据:")
14 print(df)
15 print("\n缺失值统计:")
16 print(df.isnull().sum())
17 print(f"\n总缺失值数量: {df.isnull().sum().sum()}")
18 print(f"缺失值比例: \n{df.isnull().mean() * 100}")
```

方法一：删除缺失值

1. 删除含有缺失值的行

代码块

```
1  # ===== dropna() 基本用法 =====
2
3  # 删除任何包含缺失值的行
4  df_drop_any = df.dropna()
5  print("删除所有含缺失值的行:")
6  print(df_drop_any)
7  print(f"剩余行数: {len(df_drop_any)}")
8
9  # 删除整行都是缺失值的行
10 df_drop_all = df.dropna(how='all')
11 print("\n删除全为缺失值的行:")
12 print(df_drop_all)
13
14 # 只考虑特定列的缺失值
15 df_drop_subset = df.dropna(subset=['年龄', '收入'])
16 print("\n只考虑'年龄'和'收入'列删除:")
17 print(df_drop_subset)
18
19 # 设置阈值: 至少有N个非缺失值才保留
20 df_drop_thresh = df.dropna(thresh=3) # 至少3个非空值
21 print("\n至少3个非缺失值才保留:")
22 print(df_drop_thresh)
```

2. 删除含有缺失值的列

代码块

```
1  # 删除任何包含缺失值的列
2  df_drop_col = df.dropna(axis=1)
3  print("删除所有含缺失值的列:")
4  print(df_drop_col)
5
6  # 删除整列都是缺失值的列
7  df_drop_col_all = df.dropna(axis=1, how='all')
8  print("\n删除全为缺失值的列:")
9  print(df_drop_col_all)
```

✔ 适用场景

场景	说明
缺失值很少 (< 5%)	直接删除对分析影响小
缺失完全随机	删除不会引入偏差
数据量充足	删除后仍有足够样本
关键列缺失	核心字段缺失时删除整行

✖ 不适用场景

- 缺失值比例高 (> 30%)
- 数据量本身就少
- 缺失不是随机的（有系统性原因）

🔧 方法二：填充缺失值

1. 固定值填充

代码块

```
1  # ===== fillna() 用固定值填充 =====  
2  
3  # 用0填充
```

```

4 df_fill_0 = df.fillna(0)
5 print("用0填充:")
6 print(df_fill_0)
7
8 # 用特定字符串填充
9 df_fill_str = df.fillna('未知')
10 print("\n用'未知'填充:")
11 print(df_fill_str)
12
13 # 不同列用不同值填充
14 df_fill_custom = df.fillna({
15     '年龄': 0,
16     '收入': 0,
17     '城市': '未知',
18     '评分': 0
19 })
20 print("\n不同列用不同值填充:")
21 print(df_fill_custom)

```

2. 统计值填充（最常用）

代码块

```

1 # ===== 用均值、中位数、众数填充 =====
2
3 df_fill_stats = df.copy()
4
5 # 数值列用均值填充
6 df_fill_stats['年龄'] = df_fill_stats['年龄'].fillna(df_fill_stats['年龄'].mean())
7 df_fill_stats['收入'] = df_fill_stats['收入'].fillna(df_fill_stats['收入'].mean())
8
9 # 数值列用中位数填充（对异常值更稳健）
10 df_fill_median = df.copy()
11 df_fill_median['年龄'] = df_fill_median['年龄'].fillna(df_fill_median['年龄'].median())
12 df_fill_median['收入'] = df_fill_median['收入'].fillna(df_fill_median['收入'].median())
13
14 # 分类列用众数填充

```

```

15 df_fill_mode = df.copy()
16 df_fill_mode['城市'] = df_fill_mode['城市'].fillna(df_fill_mode['城市'].mode()[0])
17 df_fill_mode['评分'] = df_fill_mode['评分'].fillna(df_fill_mode['评分'].mode()[0])
18
19 print("用均值填充:")
20 print(df_fill_stats)
21 print("\n用中位数填充:")
22 print(df_fill_median)
23 print("\n用众数填充:")
24 print(df_fill_mode)

```

3. 前后向填充（时间序列专用）

代码块

```

1  # ===== ffill (forward fill) 和 bfill (backward fill) =====
2
3  # 创建时间序列数据
4  df_time = pd.DataFrame({
5      '日期': pd.date_range('2024-01-01', periods=10, freq='D'),
6      '温度': [20, np.nan, np.nan, 23, np.nan, 25, np.nan, np.nan, 28, 30]
7  })
8
9  print("原始时间序列:")
10 print(df_time)
11
12 # 前向填充: 用前一个有效值填充
13 df_ffill = df_time.copy()
14 df_ffill['温度'] = df_ffill['温度'].ffill()
15 print("\n前向填充 (ffill):")
16 print(df_ffill)
17
18 # 后向填充: 用后一个有效值填充
19 df_bfill = df_time.copy()
20 df_bfill['温度'] = df_bfill['温度'].bfill()
21 print("\n后向填充 (bfill):")
22 print(df_bfill)
23
24 # 限制填充步数

```

```

25 df_limit = df_time.copy()
26 df_limit['温度'] = df_limit['温度'].ffill(limit=1) # 最多填充1步
27 print("\n前向填充（限制1步）：")
28 print(df_limit)

```

4. 插值填充

代码块

```

1  # ===== interpolate() 插值法 =====
2
3  df_interp = df_time.copy()
4
5  # 线性插值（默认）
6  df_interp['温度'] = df_interp['温度'].interpolate(method='linear')
7  print("线性插值：")
8  print(df_interp)
9
10 # 其他插值方法
11 df_interp_poly = df_time.copy()
12 df_interp_poly['温度'] = df_interp_poly['温
    度'].interpolate(method='polynomial', order=2)
13 print("\n多项式插值：")
14 print(df_interp_poly)

```

✓ 适用场景

填充方法	适用场景
固定值	缺失代表"无"或"未知", 如收入填0
均值	数据分布接近正态, 无极端值

中位数	数据有异常值，偏态分布
众数	分类变量（性别、城市等）
前向/后向	时间序列数据，连续观测
插值	时间序列或有序数据，平滑变化

🔑 方法三：预测填充（高级方法）

1. 基于其他特征预测

代码块

```
1  from sklearn.impute import KNNImputer
2  from sklearn.preprocessing import LabelEncoder
3
4  # ===== KNN 填充 =====
5
6  df_knn = pd.DataFrame({
7      '年龄': [25, 30, np.nan, 35, 28],
8      '收入': [5000, 6000, 7000, np.nan, 5500],
9      '消费': [2000, np.nan, 3000, 4000, 2500]
10 })
11
12 print("原始数据:")
13 print(df_knn)
14
15 # 使用KNN填充
16 imputer = KNNImputer(n_neighbors=2)
17 df_knn_filled = pd.DataFrame(
18     imputer.fit_transform(df_knn),
19     columns=df_knn.columns
20 )
21
22 print("\nKNN填充后:")
23 print(df_knn_filled.round(2))
```

2. 回归模型预测

代码块

```
1  from sklearn.linear_model import LinearRegression
2
3  # ===== 用回归模型预测缺失值 =====
4
5  df_reg = pd.DataFrame({
6      '工作经验': [1, 2, 3, 4, 5, 6],
7      '学历评分': [3, 4, 5, 4, 5, 6],
8      '薪资': [5000, 6000, np.nan, 9000, np.nan, 13000]
9  })
10
11 print("原始数据:")
12 print(df_reg)
13
14 # 分离有缺失和无缺失的数据
15 df_not_null = df_reg[df_reg['薪资'].notnull()]
16 df_is_null = df_reg[df_reg['薪资'].isnull()]
17
18 # 训练模型
19 X = df_not_null[['工作经验', '学历评分']]
20 y = df_not_null['薪资']
21 model = LinearRegression()
22 model.fit(X, y)
23
24 # 预测缺失值
25 X_missing = df_is_null[['工作经验', '学历评分']]
26 predicted_salary = model.predict(X_missing)
27
28 # 填充
29 df_reg_filled = df_reg.copy()
30 df_reg_filled.loc[df_is_null.index, '薪资'] = predicted_salary
31
32 print("\n回归预测填充后:")
33 print(df_reg_filled.round(2))
```


✓ 适用场景

- 缺失值与其他特征相关性强
- 数据量较大，可以训练模型
- 需要较高精度的填充
- 计算资源充足

🔧 方法四：标记缺失值

代码块

```
1  # ===== 保留缺失信息，添加指示列 =====
2
3  df_marked = df.copy()
4
5  # 为每列添加是否缺失的标记
6  for col in ['年龄', '收入', '城市', '评分']:
7      df_marked[f'{col}_缺失'] = df_marked[col].isnull().astype(int)
8
9  # 然后用某种方式填充原列
10 df_marked['年龄'] = df_marked['年龄'].fillna(df_marked['年龄'].median())
11 df_marked['收入'] = df_marked['收入'].fillna(df_marked['收入'].median())
12
13 print("添加缺失标记后:")
14 print(df_marked)
```

✓ 适用场景

- 缺失本身包含信息（如"不填收入"可能代表低收入）

- 机器学习建模时
- 需要分析缺失模式

方法选择决策树

代码块

```
1  缺失值处理决策流程：
2  |
3  |— 缺失比例 < 5%?
4  |   |— 是 → 直接删除 (dropna)
5  |   |— 否 ↓
6  |
7  |— 数据是时间序列?
8  |   |— 是 → 前向/后向填充 (ffill/bfill) 或插值
9  |   |— 否 ↓
10 |
11 |— 缺失值是分类变量?
12 |   |— 是 → 用众数填充 或 标记为"未知"
13 |   |— 否 ↓
14 |
15 |— 数据有异常值?
16 |   |— 是 → 用中位数填充
17 |   |— 否 → 用均值填充
18 |
19 |— 缺失与其他特征相关?
20 |   |— 是 → 用模型预测填充 (KNN/回归)
21 |   |— 否 ↓
22 |
23 |— 缺失本身有意义?
24 |   |— 是 → 添加缺失标记列 + 填充
25 |   |— 否 → 根据业务逻辑选择上述方法
```

代码块

```
1  import pandas as pd
2  import numpy as np
3
4  # 创建真实的员工数据
5  np.random.seed(42)
6  n = 100
7
8  df_employee = pd.DataFrame({
9      '员工ID': range(1, n+1),
10     '年龄': np.random.randint(22, 60, n).astype(float),
11     '工龄': np.random.randint(0, 30, n).astype(float),
12     '薪资': np.random.randint(5000, 30000, n).astype(float),
13     '部门': np.random.choice(['技术', '销售', '人事', '财务'], n),
14     '绩效': np.random.uniform(1, 5, n)
15 })
16
17 # 人为制造缺失值
18 missing_indices_age = np.random.choice(n, 10, replace=False)
19 missing_indices_salary = np.random.choice(n, 15, replace=False)
20 missing_indices_dept = np.random.choice(n, 8, replace=False)
21
22 df_employee.loc[missing_indices_age, '年龄'] = np.nan
23 df_employee.loc[missing_indices_salary, '薪资'] = np.nan
24 df_employee.loc[missing_indices_dept, '部门'] = np.nan
25
26 print("="*70)
27 print("员工数据处理完整流程")
28 print("="*70)
29
30 # 1. 检查缺失情况
31 print("\n【步骤1】缺失值分析")
32 print("-"*70)
33 print(df_employee.isnull().sum())
34 print(f"\n缺失比例:")
35 print((df_employee.isnull().sum() / len(df_employee) * 100).round(2))
36
37 # 2. 处理策略
38 print("\n【步骤2】应用不同的处理策略")
39 print("-"*70)
40
41 df_processed = df_employee.copy()
42
```

```

43 # 数值列：用中位数填充（避免异常值影响）
44 df_processed['年龄'] = df_processed['年龄'].fillna(df_processed['年龄'].median())
45 df_processed['薪资'] = df_processed['薪资'].fillna(df_processed['薪资'].median())
46
47 # 分类列：用众数填充
48 df_processed['部门'] = df_processed['部门'].fillna(df_processed['部门'].mode()
49 [0])
50
51 print("处理后的缺失值统计:")
52 print(df_processed.isnull().sum())
53
54 # 3. 验证结果
55 print("\n【步骤3】验证处理结果")
56 print("-"*70)
57 print(f"原始数据形状: {df_employee.shape}")
58 print(f"处理后数据形状: {df_processed.shape}")
59 print(f"是否还有缺失值: {df_processed.isnull().any().any()}")
60
61 # 4. 查看统计信息
62 print("\n【步骤4】数值列统计对比")
63 print("-"*70)
64 print("年龄 - 原始中位数:", df_employee['年龄'].median())
65 print("年龄 - 填充后中位数:", df_processed['年龄'].median())
66 print("\n薪资 - 原始中位数:", df_employee['薪资'].median())
67 print("薪资 - 填充后中位数:", df_processed['薪资'].median())
68
69 print("\n✅ 缺失值处理完成!")

```

最佳实践建议

1. 处理前先分析

代码块

```

1 # 了解缺失模式
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4

```

```
5 plt.figure(figsize=(10, 6))
6 sns.heatmap(df.isnull(), cbar=False, yticklabels=False)
7 plt.title('缺失值可视化')
8 plt.show()
```

2. 不要盲目删除

- 先分析缺失原因
- 评估删除对分析的影响
- 考虑是否有替代方案

3. 选择合适的填充方法

- 数值型：中位数 > 均值（更稳健）
- 分类型：众数或"未知"
- 时间序列：ffill/bfill 或插值

4. 记录处理过程

代码块

```
1  # 保存处理前的状态
2  df_original = df.copy()
3
4  # 记录处理日志
5  missing_log = {
6      '列名': [],
7      '缺失数': [],
8      '处理方法': []
9  }
```

5. 验证处理效果

```
1 # 检查是否还有缺失
2 assert df_processed.isnull().sum().sum() == 0, "仍有缺失值！"
3
4 # 检查数据分布是否合理
5 print(df_processed.describe())
```

方法对比总结表

方法	优点	缺点	适用场景
删除	简单快速	损失数据	缺失<5%，数据量大
固定值	实现简单	可能引入偏差	缺失有明确含义
均值	保持均值不变	受异常值影响	正态分布数据
中位数	抗异常值	改变分布形态	偏态分布数据
众数	适合分类	可能过度集中	分类变量
ffill/bfill	保持趋势	仅适合时序	时间序列
插值	平滑自然	计算复杂	有序连续数据
KNN	利用相关性	计算量大	特征相关性强
回归	精度高	可能过拟合	有强预测因子
标记	保留信息	增加维度	缺失有意义

根据具体数据和业务场景选择最合适的方法！